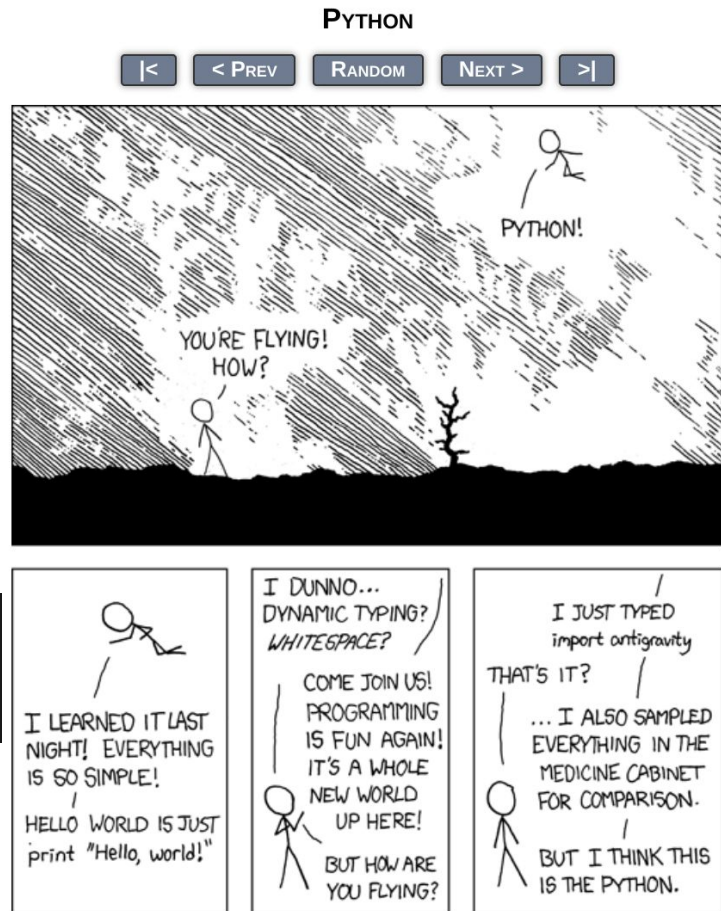


We start at 8:30

Python for HPC

Andrea Zonca | SI26



What you will be able to do

- Compile and benchmark a numerical Python loop with Numba.
- Choose threads or processes by reasoning about the GIL and overhead.
- Choose Dask chunks and scale one expression from one node to multiple nodes.

How today works

- Core path: one Numba exercise, one scheduler exercise, one Dask capstone.
- Deep dives are labeled optional and can be completed later.
- Blue sticky note: I am stuck or need help.
- Yellow sticky note: I am ready to continue.

Start with evidence

- Verify the answer.
- Profile or time a baseline.
- Change one layer: compile, schedule, chunk, or distribute.
- Verify again.
- Compare steady-state time and resources.

Numba JIT

- `@njit` turns a supported Python function into machine code.
- The first call compiles a signature.
- Warm up before timing.
- Numba helps loops that do not vectorize cleanly.

Hands-on 1

- Complete `sum_of_squares`.
- Check it against NumPy.
- Warm up the compiled function.
- Time both implementations.
- Explain the result in one sentence.
- 12 min. Blue: help. Yellow: ready.

Threads and processes

- Threads share memory and one CPython interpreter.
- The GIL limits simultaneous pure-Python bytecode.
- Processes have separate interpreters and memory.
- Data transfer and startup are not free.

Hands-on 2: predict first

- CPU-bound pure Python: threads or processes?
- Waiting on I/O: threads or processes?
- Use at most eight workers.
- Explain any surprise.
- 12 min. Blue: help. Yellow: ready.

Dask: graph then execution

- Delayed functions create tasks.
- Dependencies form a directed acyclic graph.
- A scheduler decides when and where tasks run.
- `compute()` triggers execution.

Choose useful chunks

- Too small: scheduling overhead dominates.
- Too large: poor parallelism and memory pressure.
- Useful chunks fit memory and keep workers busy.
- Inspect `.chunks`, `.numblocks`, and `.nbytes`.

Scale across nodes

- Notebook: builds the graph and requests the result.
- Scheduler: tracks dependencies and assigns tasks.
- Workers: hold chunks and execute tasks.
- The Dask array expression does not change.

Capstone: run the cluster

- Start the scheduler in the Jupyter terminal.
- Submit the two-node worker job from the login node.
- Connect with the scheduler file.
- Verify workers, compute, then cancel the job.
- 18 minutes. Blue means help. Yellow means ready.

AI as reviewer

- State environment, limits, and correctness.
- Ask for one small, explained change.
- Inspect commands and resources.
- Test correctness, then speed.
- Keep only verified improvements.

AI: ask, inspect, test

- Ask for one Numba change.
- Identify assumptions.
- Exclude compilation from timing.
- Test correctness and benchmark twice.
- Decide: accept, revise, or reject.
- 8 min. Explain your decision.

Choose one layer

- Hot numerical loop: Numba.
- Independent waiting tasks: threads or delayed.
- CPU-bound Python tasks: processes.
- Chunked array: Dask array.
- More than one node: distributed scheduler.

What you take home

- Tested core and optional notebooks.
- SI26 SLURM and Galyleo templates.
- A debug-queue validation workflow.
- A repeatable AI review checklist.
- A decision map for your next slow program.